

Guía Pedagógica: Día 9 - Favoritos y Persistencia (SSR Safe)

Hoy aprenderemos a persistir datos en el navegador del usuario utilizando **LocalStorage**. Esto permitirá que las películas marcadas como favoritas se mantengan guardadas incluso si el usuario cierra el navegador o refresca la página.

Además, enfrentaremos un reto común en el desarrollo moderno: el **SSR (Server Side Rendering)**. Aprenderemos a escribir código "seguro para el servidor", asegurándonos de que nuestro acceso al LocalStorage no rompa la aplicación cuando se ejecute fuera del navegador.

Contexto Pedagógico: El Reto del SSR

Imagina que tienes un libro de recetas (LocalStorage).

1. Si estás en tu cocina (Navegador), puedes abrir el libro y leerlo.
2. Si estás hablando por teléfono con un amigo (Servidor), él no puede ver tu libro físico. Si intentas decirle "lee la página 5", él se confundirá porque no tiene el libro.

En Angular SSR, el código se ejecuta tanto en el servidor como en el navegador. Como localStorage es una característica del navegador, si el servidor intenta leerlo, la aplicación fallará. Por eso usaremos isPlatformBrowser.

Paso 1: El Servicio de Favoritos (Global State)

Crearemos un servicio que centralice toda la lógica de favoritos. Usaremos una **Signal** para que cualquier componente que use este servicio se actualice automáticamente cuando la lista cambie.

Abre **src/app/core/services/favorites.service.ts**:

```

import { Injectable, PLATFORM_ID, inject, signal } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';
import { Movie } from '../models/movie.model';

@Injectable({ providedIn: 'root' })
export class FavoritesService {
  private platformId = inject(PLATFORM_ID);
  private readonly STORAGE_KEY = 'movienexus_favorites';

  // Nuestra "fuente de la verdad" reactiva
  public favorites = signal<Movie[]>([]);

  constructor() {
    this.loadFavorites();
  }

  private loadFavorites(): void {
    // IMPORTANTE: Solo leemos LocalStorage si estamos en el navegador
    if (isPlatformBrowser(this.platformId)) {
      const stored = localStorage.getItem(this.STORAGE_KEY);
      if (stored) this.favorites.set(JSON.parse(stored));
    }
  }

  toggleFavorite(movie: Movie): void {
    const isAlreadyFavorite = this.favorites().some(f => f.id === movie.id);
    const updated = isAlreadyFavorite
      ? this.favorites().filter(f => f.id !== movie.id)
      : [movie, ...this.favorites()];

    this.favorites.set(updated);

    // Guardamos en disco (localStorage)
    if (isPlatformBrowser(this.platformId)) {
      localStorage.setItem(this.STORAGE_KEY, JSON.stringify(updated));
    }
  }

  isFavorite(movieId: number): boolean {
    return this.favorites().some(f => f.id === movieId);
  }
}

```

Paso 2: El Botón de Corazón (MovieCard)

Queremos que el usuario pueda marcar favoritos desde cualquier listado.

1. Lógica (TypeScript)

En movie-card.ts, inyectamos el servicio y creamos el método toggleFavorite.

```
private favoritesService = inject(FavoritesService);

get isFavorite(): boolean {
  return this.favoritesService.isFavorite(this.movie.id);
}

toggleFavorite(event: Event) {
  event.preventDefault(); // Evita navegar al detalle al dar click al corazón
  event.stopPropagation();
  this.favoritesService.toggleFavorite(this.movie);
}
```

2. Estilos (CSS)

En movie-card.css, creamos un botón flotante con un efecto de latido o cambio de color.

```
.favorite-btn {
  position: absolute;
  top: 10px;
  right: 10px;
  background: rgba(0, 0, 0, 0.5);
  border-radius: 50%;
  /* ... más estilos en el archivo final ... */
}

.favorite-btn.active .heart-icon {
  fill: #e50914; /* Rojo intenso */
}
```

Paso 3: La Página de Mis Favoritos

Crearemos una nueva sección donde el usuario pueda ver toda su colección.

1. La Lógica (TypeScript)

Abre **src/app/features/favorites/favorites.ts**, inyectamos el servicio para exponer la Signal a la vista.

```
import { Component, inject } from '@angular/core';
import { FavoritesService } from '../../core/services/favorites.service';

@Component({
  // ... selector, standalone, etc.
})
export class Favorites {
  // Inyectamos el servicio para acceder a los favoritos
  public favoritesService = inject(FavoritesService);

  // Creamos un getter para facilitar el acceso en el HTML
  get favoriteMovies() {
    return this.favoritesService.favorites();
  }
}
```

La Vista (HTML)

En **src/app/features/favorites/favorites.html**, usamos el nuevo flujo de control de Angular para mostrar las tarjetas o un mensaje si no hay nada.

```
@if (favorites().length > 0) {
  <div class="movies-grid">
    @for (movie of favorites(); track movie.id) {
      <app-movie-card [movie]="movie"></app-movie-card>
    }
  </div>
} @else {
  <p>No tienes películas guardadas aún. ❤️</p>
}
```

Mis Favoritos

Tus películas guardadas para ver más tarde



Tu lista está vacía

Aún no has guardado ninguna película. ¡Explora el catálogo y añade tus favoritas!

[Explorar Películas](#)

Mis Favoritos

Tus películas guardadas para ver más tarde



Prueba de Aprendizaje (Checklist)

1. **¿Por qué no podemos usar localStorage directamente en el constructor de un servicio en Angular SSR?** *(Respuesta: Porque el servidor no tiene LocalStorage. Intentar acceder a él fuera del navegador provocará un error de "localStorage is not defined").*
2. **¿Qué función de Angular nos permite verificar si el código se está ejecutando en el cliente (navegador)?** *(Respuesta: La función isPlatformBrowser(platformId)).*
3. **¿Qué ventaja tiene usar una Signal para la lista de favoritos en lugar de un array normal?** *(Respuesta: Las Signals son reactivas. Si añado un favorito en la página de detalles, la página de "Mis Favoritos" y los corazones de las tarjetas en la Home se actualizarán instantáneamente sin necesidad de recargar nada).*

Control de Versiones

Sube el avance:

```
git add .
git commit -m "feat: implementar sistema de favoritos con persistencia localStorage y SSR safety"
git push origin main
```